

ECE 315

Steven Knudsen, PhD, PEng

Lecture 25 :

- DMA Example (RP2040)
- Programmable I/O (PIO)



UNIVERSITY
OF ALBERTA

Last Lecture

- Direct Memory Access (DMA)

This Lecture

- DMA Example (RP2040)
- Programmable I/O (PIO)

Learning Objectives

- Understand and Explain DMA Fundamentals
 - Example each part of a simple memory-to-memory transfer example
- Configure and Use DMA on the RP2040
- Understand the Purpose and Architecture of Programmable I/O (PIO) on the RP2040
- Understand PIO Programming Concepts
 - Explain the Operation of PIO Shift Registers and FIFOs
 - Understand the 9 assembly instructions and simple examples of their use
 - How DMA might be used with PIO to completely off-load the CPU

Simple Memory-to-Memory DMA

- This example is straight from the Pico example files inside the C/C++ SDK
- One reason to use it instead of something more complicated is that Wokwi only supports Pico DMA with PIO
- We will look at PIO (Programmable I/O) later...
- This example sets a DMA channel to copy a block of memory in RAM to another location.

DMA Example

```
/**
 * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
 *
 * SPDX-License-Identifier: BSD-3-Clause
 */

// Use the DMA to copy data between two buffers in memory.

#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/dma.h"

// Data will be copied from src to dst
const char src[] = "Hello, world! (from DMA)";
char dst[count_of(src)];

int main() {
    stdio_init_all();

    // Get a free channel, panic() if there are none
    int chan = dma_claim_unused_channel(true);
```

```
    // 8 bit transfers. Both read and write address increment after each
    // transfer (each pointing to a location in src or dst respectively).
    // No DREQ is selected, so the DMA transfers as fast as it can.

    dma_channel_config c = dma_channel_get_default_config(chan);
    channel_config_set_transfer_data_size(&c, DMA_SIZE_8);
    channel_config_set_read_increment(&c, true);
    channel_config_set_write_increment(&c, true);

    dma_channel_configure(
        chan,          // Channel to be configured
        &c,            // The configuration we just created
        dst,           // The initial write address
        src,           // The initial read address
        count_of(src), // Number of transfers; in this case each is 1 byte.
        true           // Start immediately.
    );

    // We could choose to go and do something else whilst the DMA is doing its
    // thing. In this case the processor has nothing else to do, so we just
    // wait for the DMA to finish.
    dma_channel_wait_for_finish_blocking(chan);

    // The DMA has now copied our text from the transmit buffer (src) to the
    // receive buffer (dst), so we can print it out from there.
    puts(dst);
}
```



DMA Example

```
/**
 * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
 *
 * SPDX-License-Identifier: BSD-3-Clause
 */

// Use the DMA to copy data between two buffers in memory.

#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/dma.h"

// Data will be copied from src to dst
const char src[] = "Hello, world! (from DMA)";
char dst[count_of(src)];

int main() {
    stdio_init_all();

    // Get a free channel, panic() if there are none
    int chan = dma_claim_unused_channel(true);

    // 8 bit transfers. Both read and write address increment after each
    // transfer (each pointing to a location in src or dst respectively).
    // No DREQ is selected, so the DMA transfers as fast as it can.

    dma_channel_config c = dma_channel_get_default_config(chan);
    channel_config_set_transfer_data_size(&c, DMA_SIZE_8);
    channel_config_set_read_increment(&c, true);
    channel_config_set_write_increment(&c, true);

    dma_channel_configure(
        chan, // Channel to be configured
        &c, // The configuration we just created
        dst, // The initial write address
        src, // The initial read address
        count_of(src), // Number of transfers; in this case each is 1 byte.
        true // Start immediately.
    );

    // We could choose to go and do something else whilst the DMA is doing its
    // this case the processor has nothing else to do, so we just
    // the DMA to finish.
    wait_for_finish_blocking(chan);

    // The DMA has now copied our text from the transmit buffer (src) to the
    // receive buffer (dst), so we can print it out from there.
    puts(dst);
}
```

Get a free channel. Which one doesn't matter



DMA Example

```
/**
 * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
 *
 * SPDX-License-Identifier: BSD-3-Clause
 */

// Use the DMA to copy data between two buffers in memory.

#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/dma.h"

// Data will be copied from src to dst
const char src[] = "Hello, world! (from DMA)";
char dst[count_of(src)];

int main() {
    stdio_init_all();

    // Get a free channel, panic() if there are none
    int chan = dma_claim_unused_channel(true);

    // 8 bit transfers. Both read and write address increment after each
    // transfer (each pointing to a location in src or dst respectively).
    // No DREQ is selected, so the DMA transfers as fast as it can.

    dma_channel_config c = dma_channel_get_default_config(chan);
    channel_config_set_transfer_data_size(&c, DMA_SIZE_8);
    channel_config_set_read_increment(&c, true);
    channel_config_set_write_increment(&c, true);

    dma_channel_configure(
        chan, // Channel to be configured
        &c, // The configuration we just created
        dst, // The initial write address
        src, // The initial read address
        count_of(src), // Number of transfers; in this case each is 1 byte.
        true // Start immediately.
    );

    // We could choose to go and do something else whilst the DMA is doing its
    // this case the processor has nothing else to do, so we just
    // the DMA to finish.
    wait_for_finish_blocking(chan);

    // The DMA has now copied our text from the transmit buffer (src) to the
    // receive buffer (dst), so we can print it out from there.
}
```



Get a free channel. Which one doesn't matter

Hmmm, what is wrong with this?

8

Bad programmer, should check that the returned value is valid...

Copy and paste this into another program where lots of DMA channels are used and it may cause problems...

DMA Example

```
/**
 * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
 *
 * SPDX-License-Identifier:
 */

// Use the DMA to copy data

#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/dma.h"

// Data will be copied from
const char src[] = "Hello, world";
char dst[count_of(src)];

int main() {
    stdio_init_all();

    // Get a free channel, panic() if there are none
    int chan = dma_claim_unused_channel(true);
```

Configure the channel;

- Start with the default configuration
- Doing a memory-to-memory transfer, so for every read and write we increment the address
- Doing a byte transfer, so set the size to match

```
// 8 bit transfers. Both read and write address increment after each
// transfer (each pointing to a location in src or dst respectively).
// No DREQ is selected, so the DMA transfers as fast as it can.
```

```
dma_channel_config c = dma_channel_get_default_config(chan);
channel_config_set_transfer_data_size(&c, DMA_SIZE_8);
channel_config_set_read_increment(&c, true);
channel_config_set_write_increment(&c, true);

dma_channel_configure(
    chan, // Channel to be configured
    &dst, // The configuration we just created
    &src, // The initial write address
    &src, // The initial read address
    count_of(src), // Number of transfers; in this case each is 1 byte.
    0, // Start immediately.
```

```
// Should choose to go and do something else whilst the DMA is doing its
// thing. In this case the processor has nothing else to do, so we just
// wait for the DMA to finish.
dma_channel_wait_for_finish_blocking(chan);

// The DMA has now copied our text from the transmit buffer (src) to the
// receive buffer (dst), so we can print it out from there.
puts(dst);
}
```



WOKWI

SAVE

SHARE

ECE 315 Lecture 24 DMA Memory-to-memory

Docs

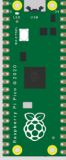
main.c

```

1 //
2 * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
3 *
4 * SPDX-License-Identifier: BSD-3-Clause
5 */
6
7 // Use the DMA to copy data between two buffers in memory.
8
9 #include <stdio.h>
10 #include "bcm2835.h"
11 #include "hardware/dma.h"
12
13 // Data will be copied from src to dst
14 const char src[] = "Hello, world! (from DMA)";
15 char dst[1024];
16
17 int main() {
18     stdio_init_all();
19
20     // Get a free channel, panic() if there are none
21     int chan = dma_claim_unused_channel(true);
22
23     // 8 bit transfers, both read and write address increment after each
24     // transfer (each pointing to a location in src or dst respectively).
25     // No DRQD is selected, so the DMA transfers as fast as it can.
26
27     dma_channel_config c = dma_channel_get_default_config(chan);
28     channel_config_set_transfer_data_size(&c, DMA_SIZE_8);
29     channel_config_set_read_increment(&c, true);
30     channel_config_set_write_increment(&c, true);
31
32     dma_channel_configure(
33         chan, // Channel to be configured
34         &c, // The configuration we just created
35         &dst, // The initial write address
36         src, // The initial read address
37         count_of(src), // Number of transfers; in this case each is 1 byte.
38         true // Start immediately.
39     );
40
41     // We could choose to go and do something else whilst the DMA is doing its
42     // thing. In this case the processor has nothing else to do, so we just
43     // wait for the DMA to finish.
44     dma_channel_wait_for_finish_blocking(chan);
45
46     // The DMA has now copied our text from the transmit buffer (src) to the
47     // receive buffer (dst), so we can print it out from there.
48     puts(dst);
49 }

```

Simulation



Hello, world! (from DMA)



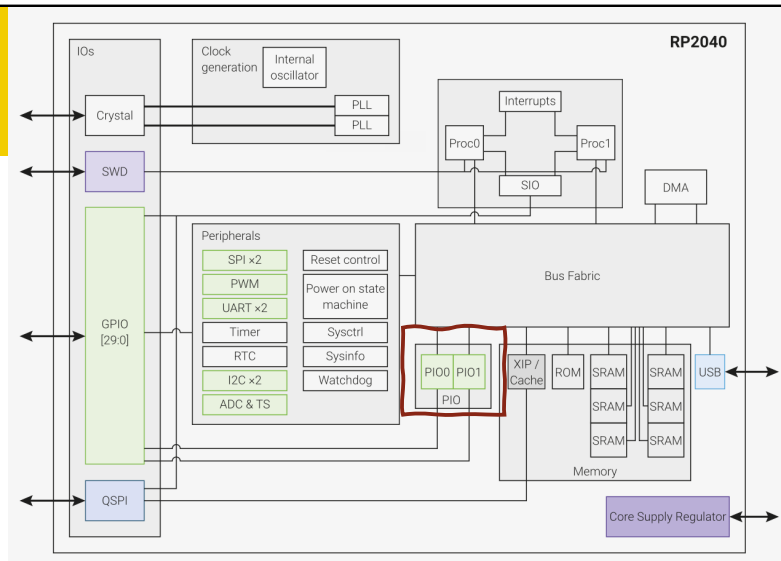
UNIVERSITY OF ALBERTA

10

Programmable Input/Output (PIO)

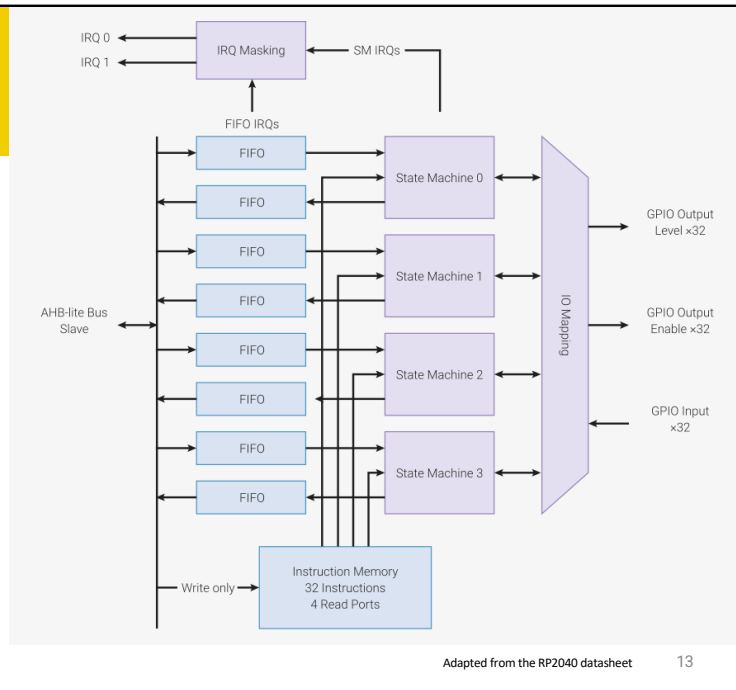
PIO Intro

- On the RP2040, Programmable Input/Output (PIO) refers to a special peripheral device on the system bus
- There are two,
 - PIO0
 - PIO1



PIO Details

- Each PIO has 4 state machines
- Each state machine
 - is mapped to input & output FIFOs
 - is mapped to 32 GPIOs
- Each is programmable using a simple pseudo-assembly language



PIO Intro cont'd

The PIO can support a variety of IO standards, including:

- 8080 and 6800 parallel bus
- I2C
- 3-pin I2S
- SDIO
- SPI, DSPI, QSPI
- UART
- DPI or VGA (via resistor DAC)

PIO Intro cont'd

The PIO can support a variety of IO standards, including:

- 8080 and 6800 parallel bus
- I2C
- 3-pin I2S
- SDIO
- SPI, DSPI, QSPI
- UART
- DPI or VGA (via resistor DAC)

!!!???

PIO Intro cont'd

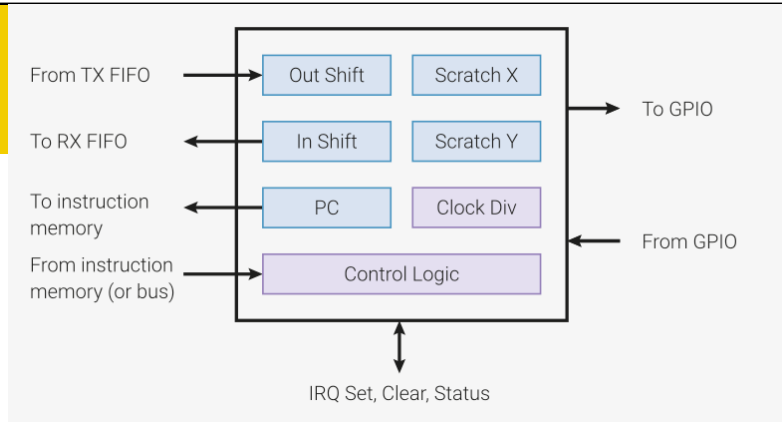
- PIO state machines are highly specialized for IO
 - supports determinism, precise timing, and close integration hardware
- Each state machine is equipped with:
 - Two 32-bit shift registers – either direction, any shift count
 - Two 32-bit scratch registers
 - 4×32-bit bus FIFO in each direction (TX/RX), reconfigurable as 8×32 in a single direction
 - Fractional clock divider (16 integer, 8 fractional bits) → super important for implementing communication protocols
 - Flexible GPIO mapping
 - DMA interface, sustained throughput up to 1 word per clock from system DMA
 - IRQ flag set/clear/status

PIO Intro cont'd

- When combined with the 32 DMA channels, some pretty complex machines can be implemented
 - Even with a relatively slow clock, moving data at 100s of MBps is possible
- As previously mentioned, Wokwi only supports Pico DMA with PIO
- Eventually we will combine what we learned about DMA with PIO for a Wokwi example

PIO Programmer's Model

- GPIO access
- An Input and Output Shift Register (ISR and OSR)
- Two scratch registers
- The program counter steps through the PIO program memory just like the PC for the CPUs
- The control logic executes the instruction referenced by the PC
- This relatively small set of primitives can be used to create complex operations



PIO Instructions aka Assembly Language

- The PIO supports just nine (9) instructions

Instruction	Definition
JMP	Branches to a different instruction, can be conditional on a few conditions (such as X or Y zero or not-zero) and decrement X or Y
WAIT	Can wait for various things like a pin changing state or an interrupt occurring
IN	shift from sources such as pin inputs or the X or Y scratch registers into the ISR
OUT	shift bits from the OSR into a destination, such as pin outputs or the X or Y scratch registers
PUSH	pushes the contents of the ISR into the input FIFO and clears the ISR. Waits if input FIFO is full
PULL	pops the front element of the output FIFO into the OSR. Waits if the output FIFO is empty
MOV	Moves from a source to a destination. Can use registers and pins as both a source and dest
IRQ	Raises an interrupt (which can interrupt one of the cores or be used to synchronize SMs)
SET	Writes an immediate value to X, Y or the pins

Simple Example

- Toggle a pin to create a square wave over 4 cycles total
- On each system clock cycle, the state machine fetches, decodes, and executes one instruction
 - Can delay up to 31 cycles before next instruction executes

```
set pins, 1 [1]
```

```
8 .program squarewave
9   set pindirs, 1   ; Set pin to output
10 again:
11   set pins, 1 [1] ; Drive pin high and then delay for one cycle
12   set pins, 0     ; Drive pin low
13   jmp again       ; Set PC to label 'again'
```

Adapted from the RP2040 datasheet

Simple Example

- Toggle a pin to create a square wave over 4 cycles total
- On each system clock cycle, the state machine fetches, decodes, and executes one instruction
 - Can delay up to 31 cycles before next instruction executes

```
set pins, 1 [1]
```

- Why does the instruction on line 12 not have a delay?

```
8 .program squarewave
9   set pindirs, 1   ; Set pin to output
10 again:
11   set pins, 1 [1] ; Drive pin high and then delay for one cycle
12   set pins, 0     ; Drive pin low
13   jmp again      ; Set PC to label 'again'
```

Adapted from the RP2040 datasheet

PIO State Machine – Program Control

- Just like a microprocessor, there is a program counter (PC) that points to PIO instructions to execute
 - Generally, the PC increments by one each clock cycle, and wraps at the end of the 32-instruction memory
 - Of course, the JMP instruction can change the PC to any of the instruction locations
- The system has write-only access to the PIO instruction memory
 - Programs can be loaded by the CPU or via DMA, the latter makes for a really powerful capability

```
34 // Load the assembled program directly into the PIO's instruction memory.
35 // Each PIO instance has a 32-slot instruction memory, which all 4 state
36 // machines can see. The system has write-only access.
37 for (uint i = 0; i < count_of(squarewave_program_instructions); ++i)
38     pio->instr_mem[i] = squarewave_program_instructions[i];
```

Adapted from the RP2040 datasheet

22

PIO State Machine – Clock Divider

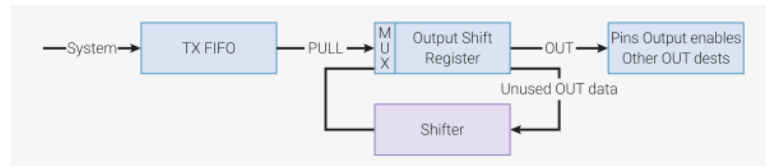
- The clock into the state machines can be scaled to effectively slow down state machine transitions
- This is accomplished using a 16.8 binary fixed-point fractional number
- If the clock divider is set to 2.5, the square wave program would have a period of $4 \times 2.5 = 10$ cycles, which at $1/125\text{e}6 = 8\text{ ns}$ would be 80 ns or 12.5 MHz

```
pio->sm[0].clkdiv = (uint32_t)(2.5f*(1 << 16));
```

- Being able to set the state machine clock is critical to achieving timing needed for things like serial protocols (UART, SPI, I2C, etc.)

PIO – Output Shift Register (OSR)

- The OSR holds and shifts output data, between the TX FIFO and the pins (or other destinations, such as the scratch registers)
 - PULL instruction removes a 32-bit word from TX FIFO and place into the OSR
 - OUT instruction shifts data from the OSR to other dest's, 1...32 bits at a time
 - Zeros (0) shifted in as data is shifted out
 - If autopull enabled, SM automatically refills the OSR from the FIFO on an OUT instruction once some total shift count threshold is reached
 - Shift direction can be left/right, configurable via configuration registers



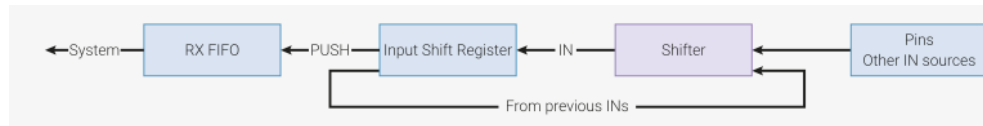
Adapted from the RP2040 datasheet



24

PIO – Input Shift Register (ISR)

- The ISR holds and shifts input data, between the RX FIFO and the pins (or other destinations, such as the scratch registers)
 - IN instruction shifts 1...32 bits at a time into the register.
 - PUSH instruction writes the ISR contents to the RX FIFO.
 - The ISR is cleared to all-zeroes when pushed.
 - If autopush enabled, SM will automatically push the ISR on an IN instruction, once some shift threshold is reached
 - Shift direction can be left/right, configurable via configuration registers



PIO – Shift Counters

- The State Machines keep track of how many bits have been shifted out of the OSR or into the ISR
 - The counters have a range of [0,32]
- The counters can be used to trigger specific actions
 - The OSR can be automatically refilled once N bits have been shifted out
 - The ISR can be automatically emptied once M bits have been shifted in
 - PUSH or PULL instructions can be conditioned on the input or output shift counter, respectively
- PIO reset or asserting CTRL_SM_RESTART causes input shift counter to be cleared and the output shift counter to be set to 32
- Other instructions, PUSH , PULL , MOV, and OUT affect the counters

PIO – Scratch Registers, FIFOs

- There are two 32-bit internal scratch registers named X and Y
- All kinds of uses for PIO programs; checking values, holding temps, ...
- Each state machine has a pair of 4-word (4x32 bit) FIFO
 - TX FIFO for transfers from the system to the state machine
 - Written to by bus masters, the CPU or DMA, for example
 - RX FIFO for transfers from the state machine to the system
 - Written to by the state machine
 - They act as buffers, decoupling the operation of the system from the PIOs
- The FIFOs can be joined to transfer data in only one direction, making for a doubly-deep FIFO

PIO – Pin Mapping

- Each PIO can set the direction and set/get the value of up to 32 GPIOs
- On each system clock cycle, none, one, or both of the following can be done
 - Change the level or direction of some GPIOs via an OUT or SET instruction, or read some GPIOs via an IN instruction
 - Change the level or direction of some GPIOs via a side-set operation
- The operations are on one of four contiguous ranges of GPIOs, which may overlap
- Generally, each state machine's outputs are mapped to a distinct group of GPIOs, implementing some peripheral interface

PIO - IRQs

- They can be set or cleared by the system or state machine
- 8 in total, visible to all 8 state machines
- Each of the lower 4 can be mapped onto one of the PIO's interrupt request lines
- State machine can use them to get the CPU to perform a task
- Can be used to synchronize operation between state machines

An Impressive Example



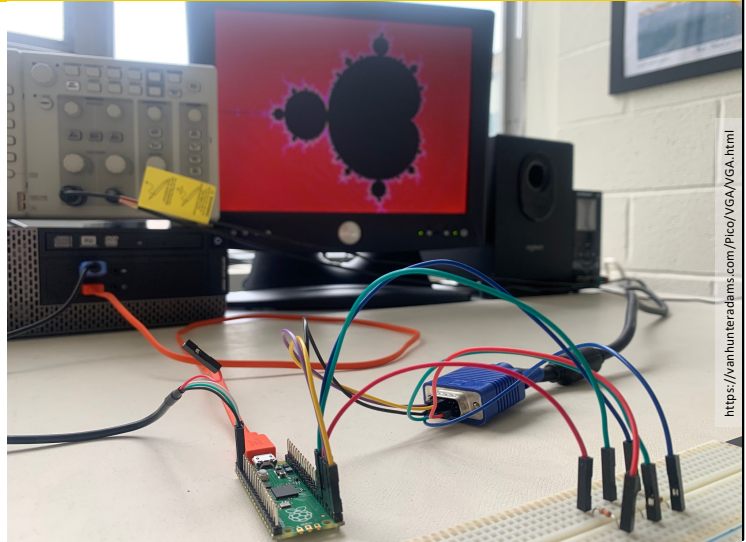
UNIVERSITY
OF ALBERTA

Adapted from the RP2040 datasheet

30

PIO Assembly VGA Driver for RP2040

- The PIO, coupled with DMA, is fast enough to create real-time VGA signals
- We will look at this next lecture and walk through a simple demo using PIO with DMA to vary an LED's brightness via pulse-width modulation



Summary

- DMA allows peripheral-to-memory, memory-to-memory, and PIO-to-memory transfers without CPU intervention
 - Reviewed a simple memory-to-memory example
- RP2040 includes two Programmable I/O (PIO) blocks (PIO0 & PIO1), each with 4 state machines, various registers, mapping to GPIOs, and system bus access
 - Can implement standard and unique comm protocols and other I/O functions
 - Supported by a simple assembly language (19 instructions) and API
- DMA + PIO enables high-bandwidth, precise, and CPU-offloaded I/O operations
- Shift registers and FIFOs enable continuous streaming of data

Next Lecture

- Pico VGA Example
- DMA + PIO Example

Questions

